PROJECT-1

3-TIER ARCHITECTURE

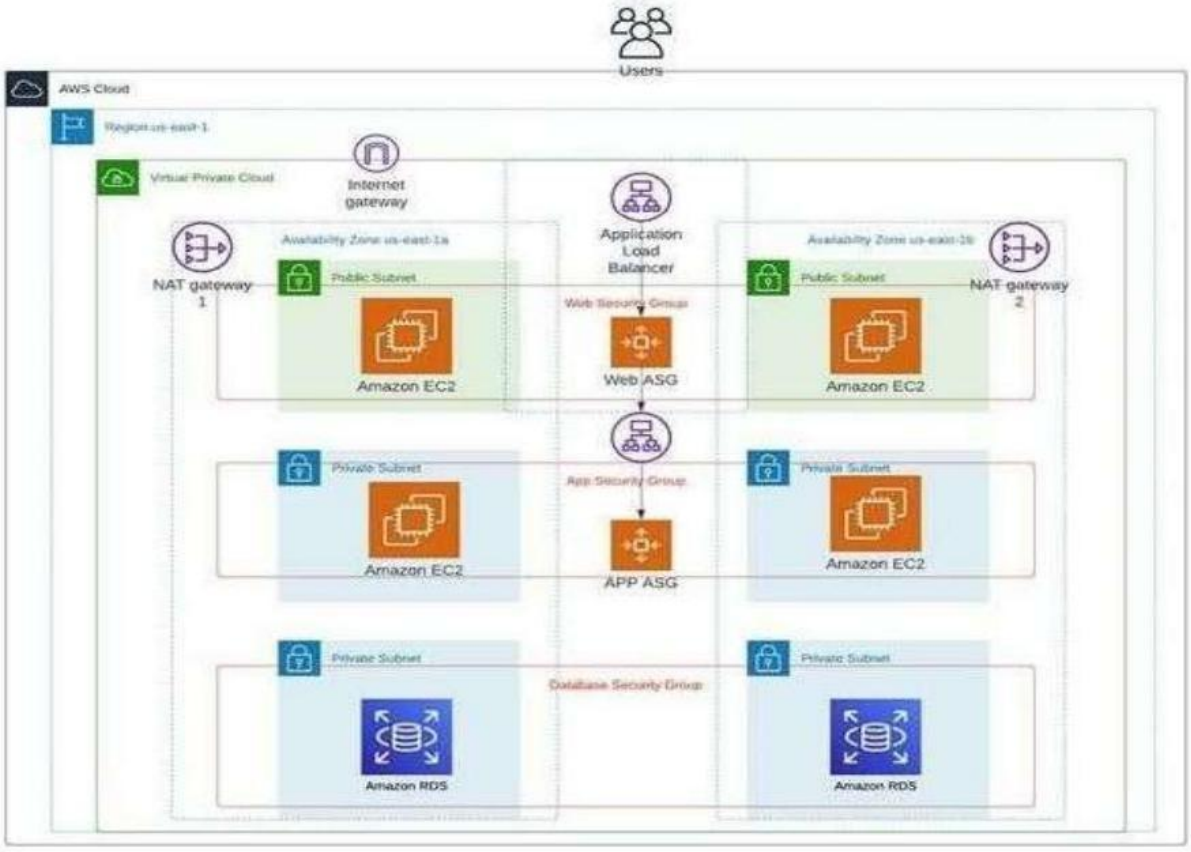
Created By : M.Karthik Reddy

Date : 13/10/2025

Batch No : 141

Institution : Vcube Softtware Solutions.

3-TIER ARICHITECTURE ON AWS



What is 3-Tier Architecture?

A 3-tier architecture is a software architecture pattern that divides an application into three logical layers:

1. Presentation Layer (Frontend/UI)

• What the user interacts with (website, web app, or mobile app).

• Example: HTML, CSS, JavaScript, React, Angular.

2. Application Layer (Business Logic / Middleware)

• Processes the user requests, applies rules, and connects to the database.

• Example: Application servers (Java, Python Flask/Django, Node.js, .NET).

3. Database Layer (Data Storage)

• Stores and manages the application's data.

• Example: MySQL, PostgreSQL, MongoDB, AWS RDS.

• This separation makes the system scalable, secure, and easy to manage.

Security Best Practices:

• Use Security Groups to allow only necessary traffic (e.g., Web→App,App → DB).

• Use Network ACLs and Subnets to segment the layers properly.

• Database should never be exposed directly to the public internet.

Open AWS Website and login to your Account

PROCESS:

Step-1: Create VPC:

a) Open VPC and Click on “Create VPC”.

b) Label the VPC (ex: Project-Vpc)

c) Specify Ipv4 CIDR block (140.0.0.0/16)

d) Create it.

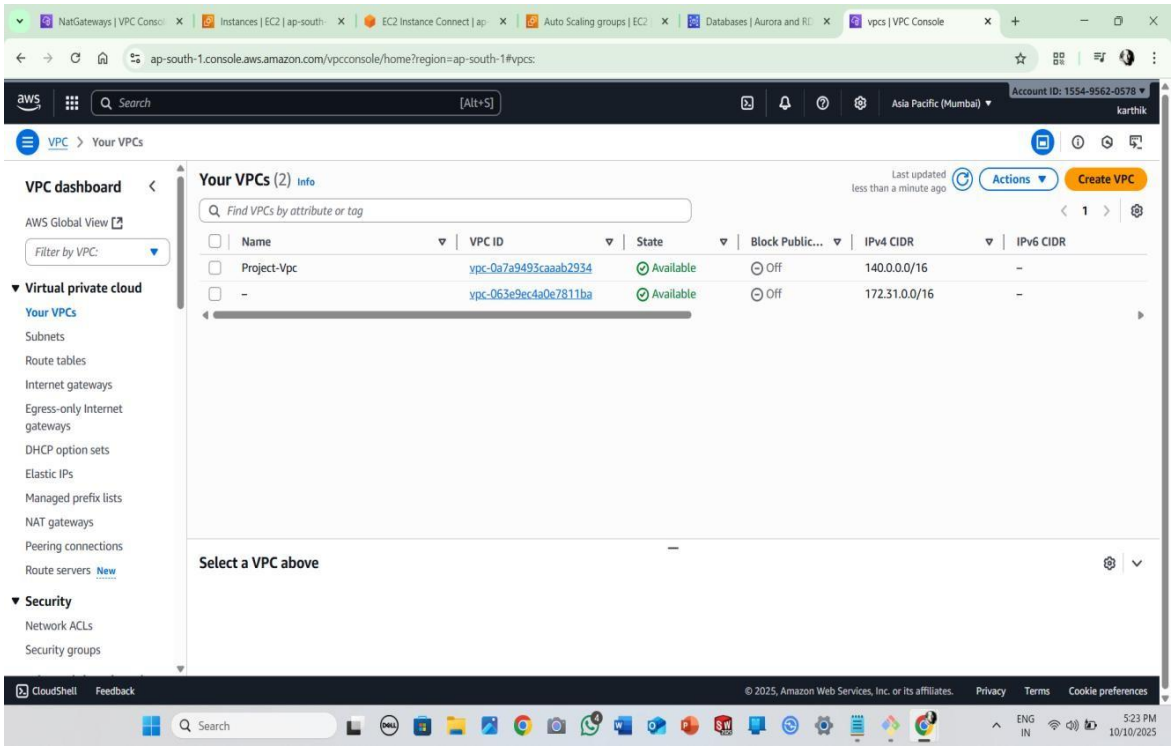


Fig-1

Step-2 : Create Subnets:

a) Click On Create Subnet.

b) Attach Vpc for subnets.

c) Create Two Public and four Private Subnets.

d) In Public Subnets Choose Different Availability Zones and Different CIDR Block.

e) In Private Subnets Choose Different AZ and CIDR Block IP Like Public

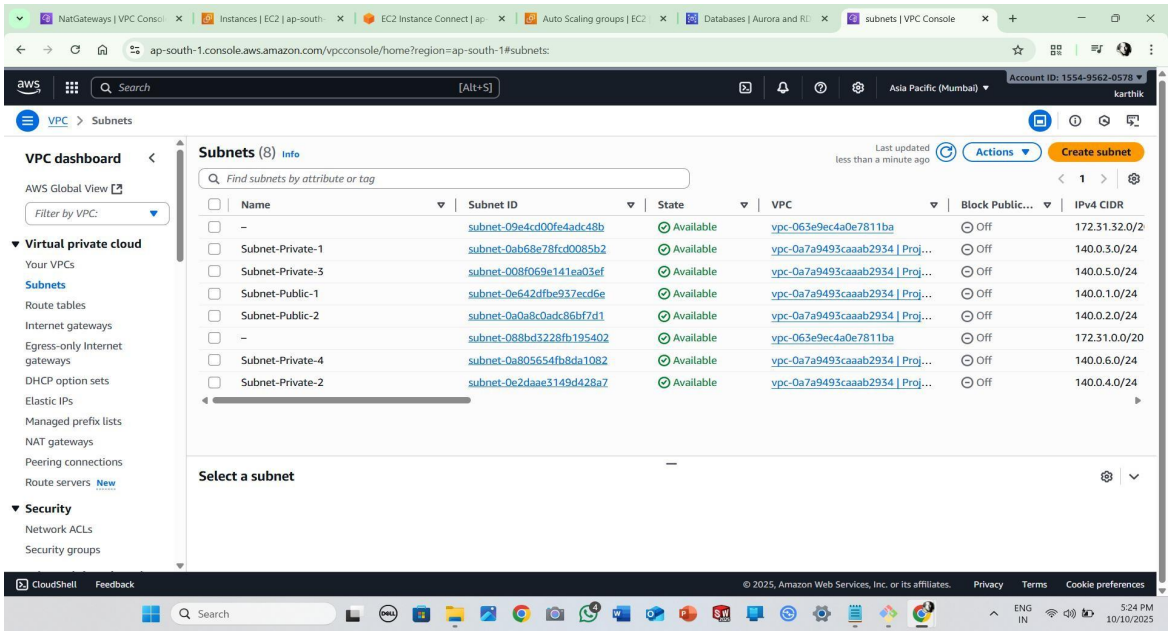


Fig-2

Step-3: Create Internet Gateway:

a) Select InternetGateway.

b) Attach Vpc to it.

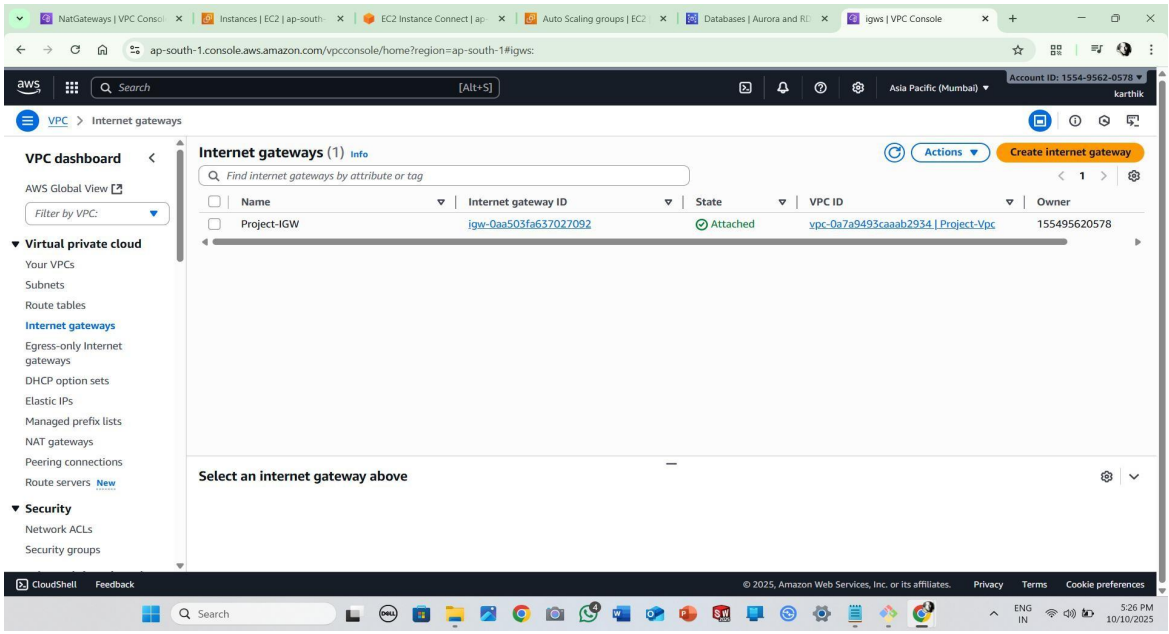


Fig-3

Step-4: Create Route Tables:

a) Open Route Tables and select main Vpc.

b) Create 2 Route Tables (public and private)

c) Attach 2 public subnets to public Route Table.

d) 4 private subnets to private Route Table.

e) Edit it and Add route (InternetGateway).

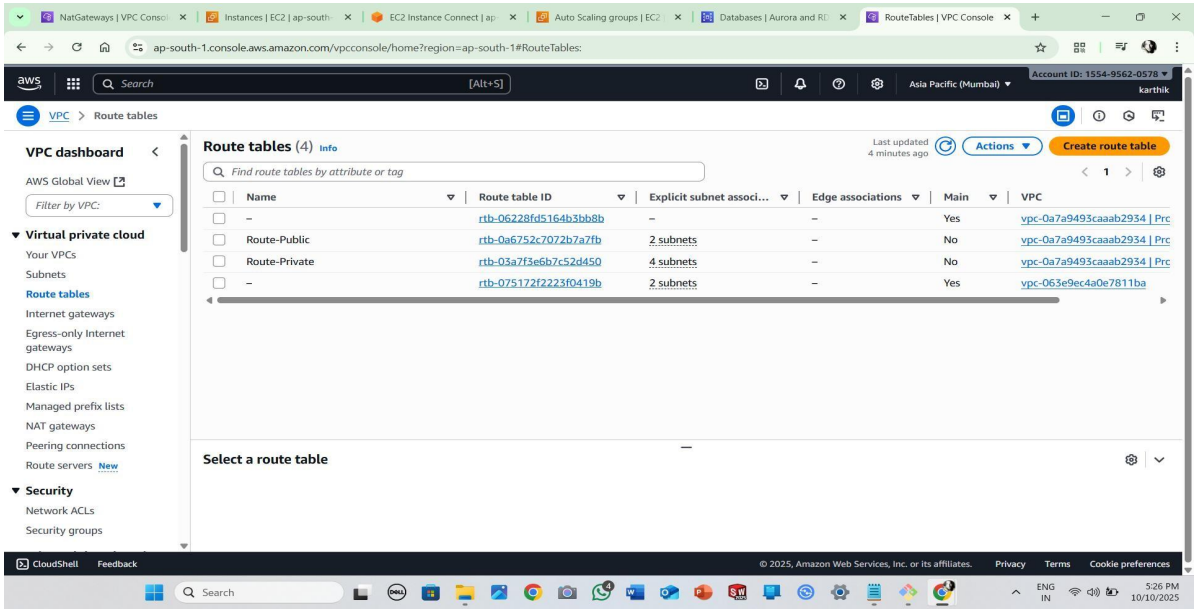


Fig-4

Step-5: Create NAT Gateway:

a) Now Open NAT Gateway.

b) Select Public Access.

c) Allocate ElasticIP and Select Subnets (2Private subnets)

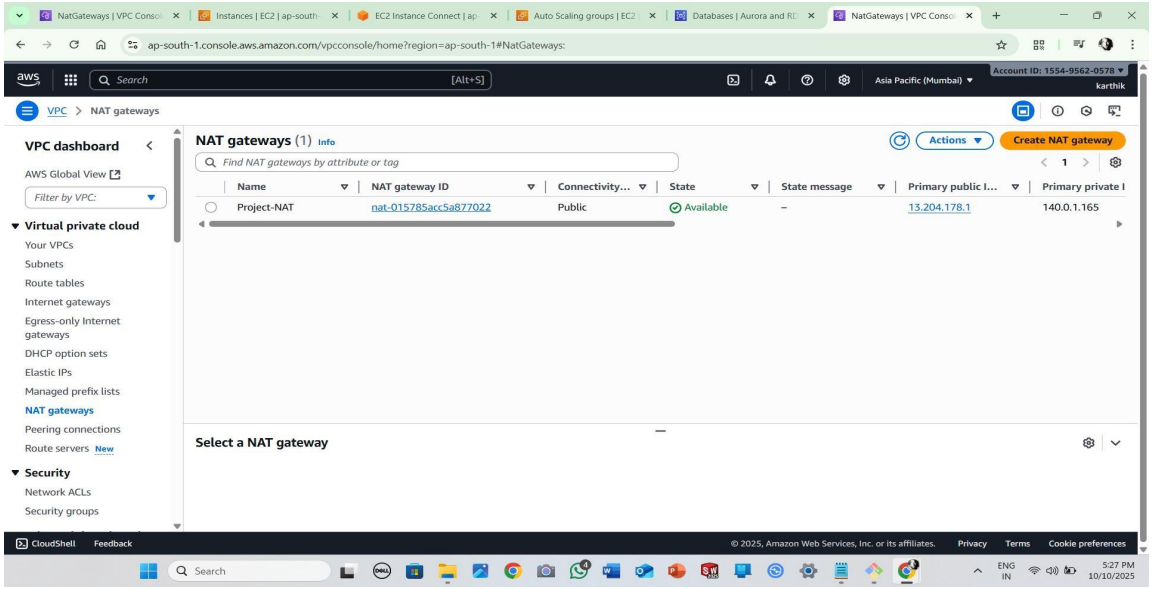


Fig-5

Step-6: Launch EC2 Instances:

a) Create 2 Public and 2 Private EC2 instances.

b) Select AMAZON LINUX & UBUNTU For all Instances.

c) Create Key Pair. (Keypair is same for every Instance).

d) Select VPC – (What We Created).

e) Add Subnet and Add Security Group.

f) Launch Instances.

g) Copy the Public Instance SSH link and paste it on Gitbash.

h) By Using Commands, we have to connect to the Server.

Step 7: Create image in Instance:

a) Click on Instance.

b) Go to Actions.

c) Click on Image and template.

d) Give A name to image and save it.

e) Create it.

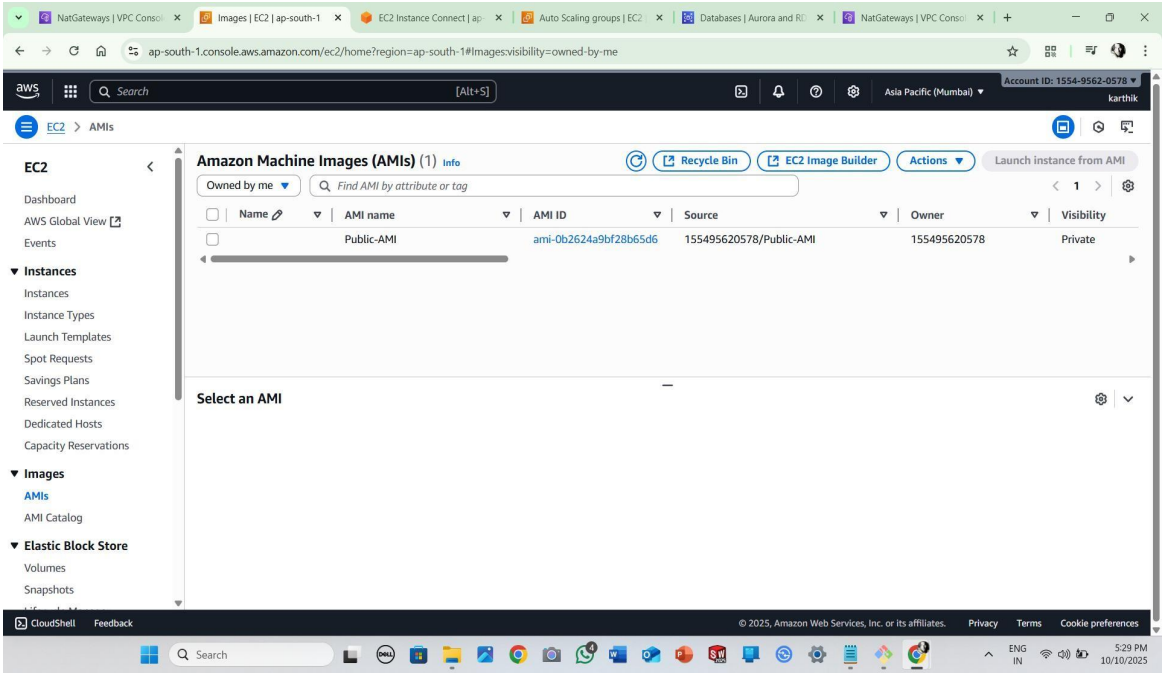


Fig-6

After Connected to the Server We Have to Commands:

a) Sudo –i
b) apt update
c) apt install nginx
d) cd /usr/share/nginx/html
e) rm index.html
f) vi index.html (insert data)
g) systemctl restart nginx(to restart the server.
h) systemctl status nginx (to see the status of server)
i) cat index.html (used to insert the data in it gitbash).

```
root@ip-172-31-33-144: ~
ubuntu@ip-172-31-33-144:~$ sudo -i
root@ip-172-31-33-144:~# apt update -y && apt install nginx -y
Hit:1 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:5 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB]
Get:6 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/universe Translation-en [5982 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1213 kB]
Get:8 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 Components [3871 kB]
Get:9 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 c-n-f Metadata [301 kB]
Get:10 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 Packages [269 kB]
Get:11 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/multiverse Translation-en [118 kB]
Get:12 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 Components [35.0 kB]
Get:13 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 c-n-f Metadata [8328 B]
Get:14 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1492 kB]
Get:15 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [202 kB]
Get:16 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.6 kB]
Get:17 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble-updates/main Translation-en [287 kB]
Get:18 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [8748 B]
Get:19 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [882 kB]
Get:20 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [175 kB]
Get:21 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [15.3 kB]
Get:22 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1489 kB]
Get:23 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [196 kB]
Get:24 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [52.2 kB]
Get:25 http://security.ubuntu.com/ubuntu noble-security/universe amd64 c-n-f Metadata [18.1 kB]
Get:26 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [1967 kB]
Get:27 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble-updates/universe Translation-en [301 kB]
Get:28 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [378 kB]
Get:29 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 c-n-f Metadata [31.1 kB]
Get:30 http://security.ubuntu.com/ubuntu noble-security/restricted Translation-en [448 kB]
Get:31 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [2068 kB]
Get:32 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B]
Get:33 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 c-n-f Metadata [520 B]
Get:34 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Packages [28.0 kB]
Get:35 http://security.ubuntu.com/ubuntu noble-security/multiverse Translation-en [5844 B]
Get:36 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [208 B]
Get:37 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 c-n-f Metadata [384 B]
```

Fig-7

```
root@ip-172-31-33-144: ~
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for ufw (0.36.2-6) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
root@ip-172-31-33-144:~# systemctl start nginx
root@ip-172-31-33-144:~# cd /usr/share/nginx/html
root@ip-172-31-33-144:/usr/share/nginx/html# rm index.html
root@ip-172-31-33-144:/usr/share/nginx/html# vi index.html
root@ip-172-31-33-144:/usr/share/nginx/html# cd
root@ip-172-31-33-144:~# systemctl restart nginx
root@ip-172-31-33-144:~# systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
     Active: active (running) since Sat 2025-10-11 08:18:21 UTC; 8s ago
       Docs: man:nginx(8)
    Process: 1895 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
    Process: 1897 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
   Main PID: 1898 (nginx)
      Tasks: 3 (limit: 1008)
     Memory: 2.4M (peak: 2.7M)
        CPU: 20ms
     CGroup: /system.slice/nginx.service
             ├─1898 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             ├─1899 "nginx: worker process"
             └─1900 "nginx: worker process"

Oct 11 08:18:21 ip-172-31-33-144 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Oct 11 08:18:21 ip-172-31-33-144 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
root@ip-172-31-33-144:~#
```

Fig-8

Step-7: Create TargetGroup:

a) Create One Public TargetGroup.

b) Give name as (public-Tg)

c) Add Public Instances to Public TG.

d) Create One Private TargetGroup.

e) Give name as (Private-Tg)

f) Add Private Instances to Private TG.

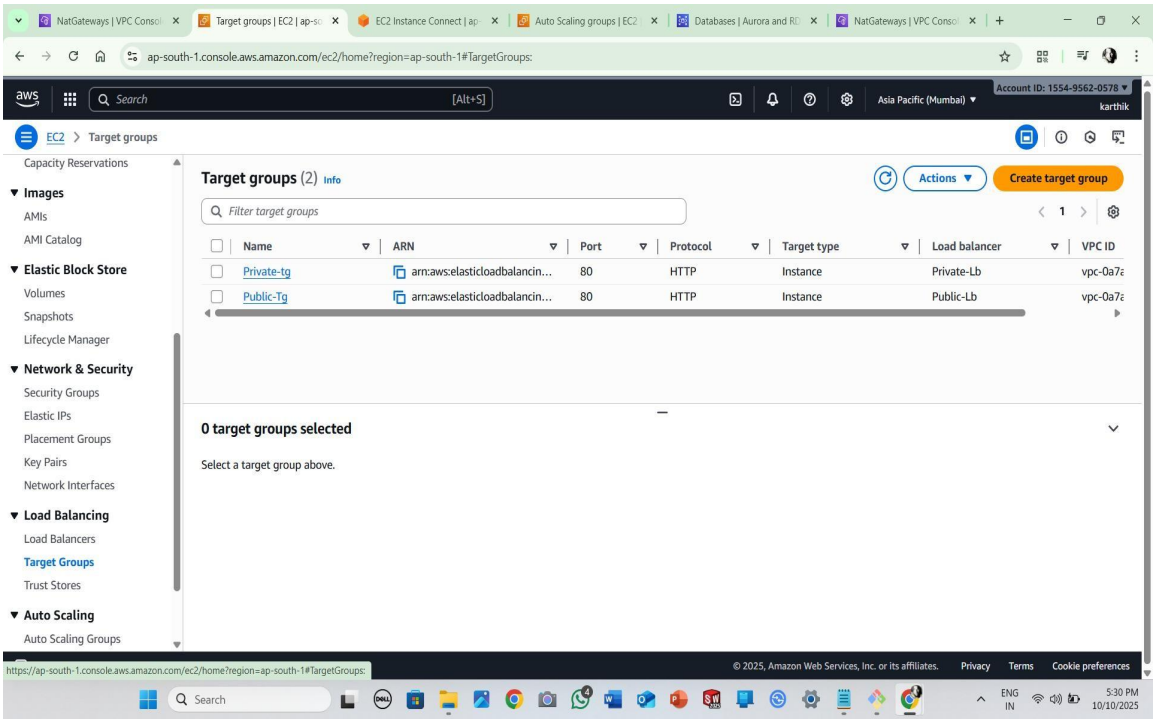


Fig-9

Step-8: Create LoadBalancer:

a) Open LoadBalancer.

b) We have to create Public and private LoadBalancer.

c) Create Public LoadBalancer.

d) Select Internet Facing.

e) Select Subnets.

f) Select our SecurityGroup. (what we created).

g) Add Target Group (we Created one).

h) Create Private Load Balancer same as Public LB

i) Click On Create LoadBalancer.

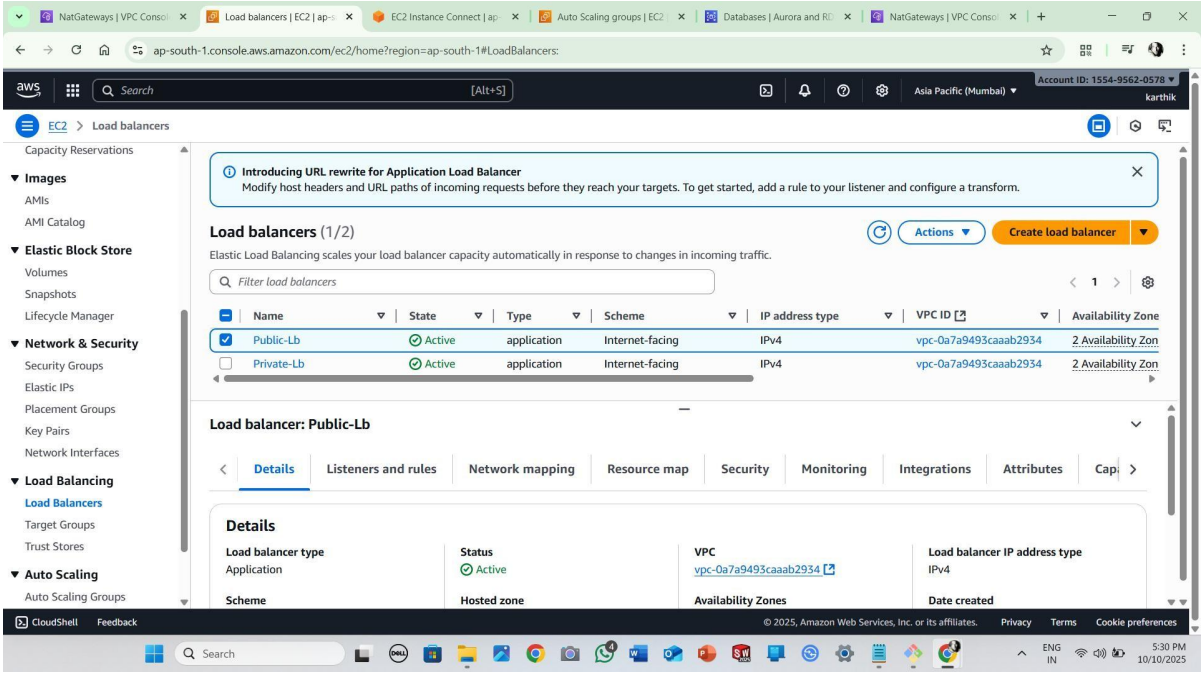


Fig-10

Step-9: AWS Auto Scaling:

a) Click on Create Auto Scaling.

b) Auto Scaling group.

c) Give Scaling Policies.

d) Give the size (max or min).

e) Attach the load Balancer. (what we created).

f) Create Public and Private AutoScaling.

g) Add Public Subnet to Public AutoScaling.

h) Add Private Subnet to Private AutoScaling.

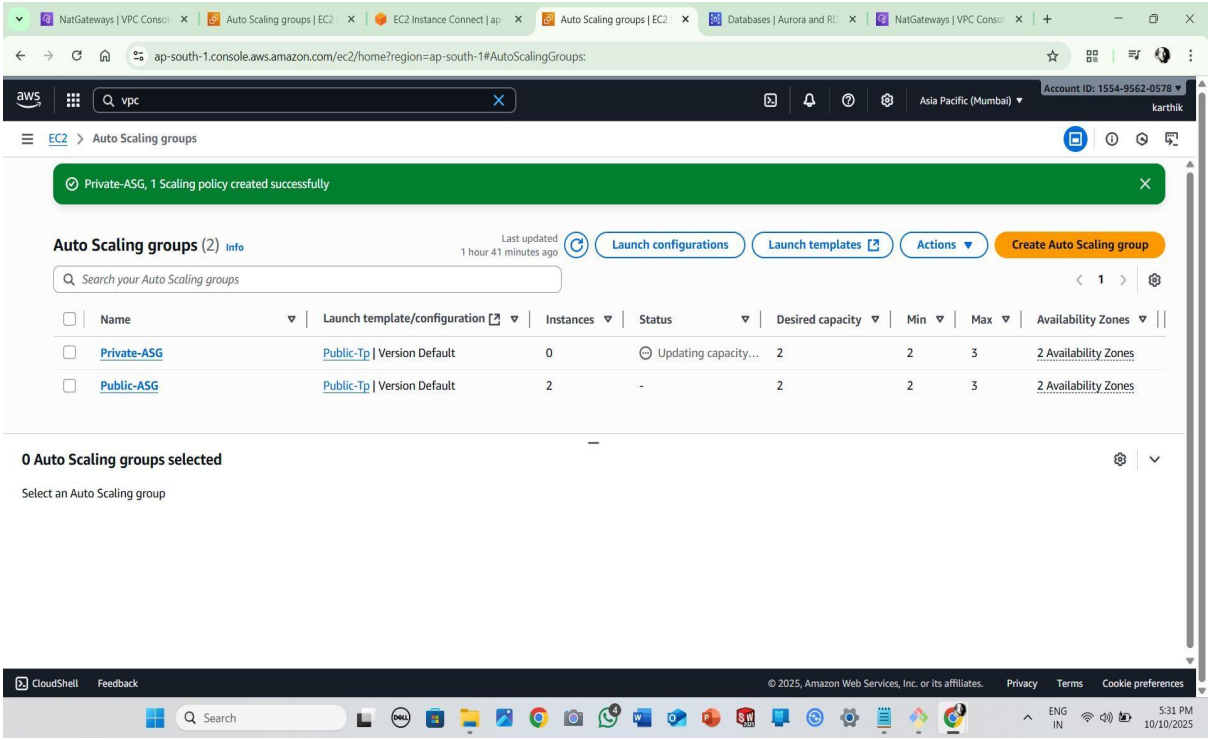


Fig- 11

i) In EC2 Instance we can see the New Instances Created.

J) If we add subnets in auto Scaling. It can create EC2 Instances in EC2.

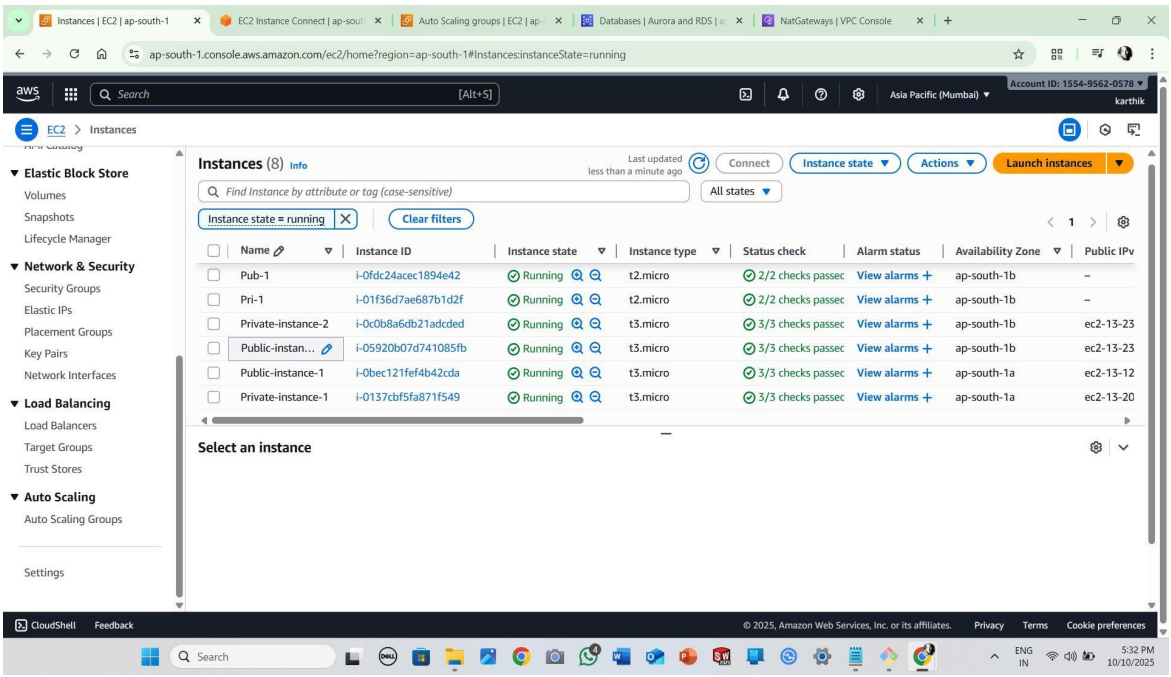


Fig-12

Step-10: Create Database In RDS :

a) Open Arora and RDS.

b) Click On Databases.

c) Choose Engine as: MySQL.

d) In Credentials Management choose myself.

e) Give new password to it.

f) It shows monthly Estimated.

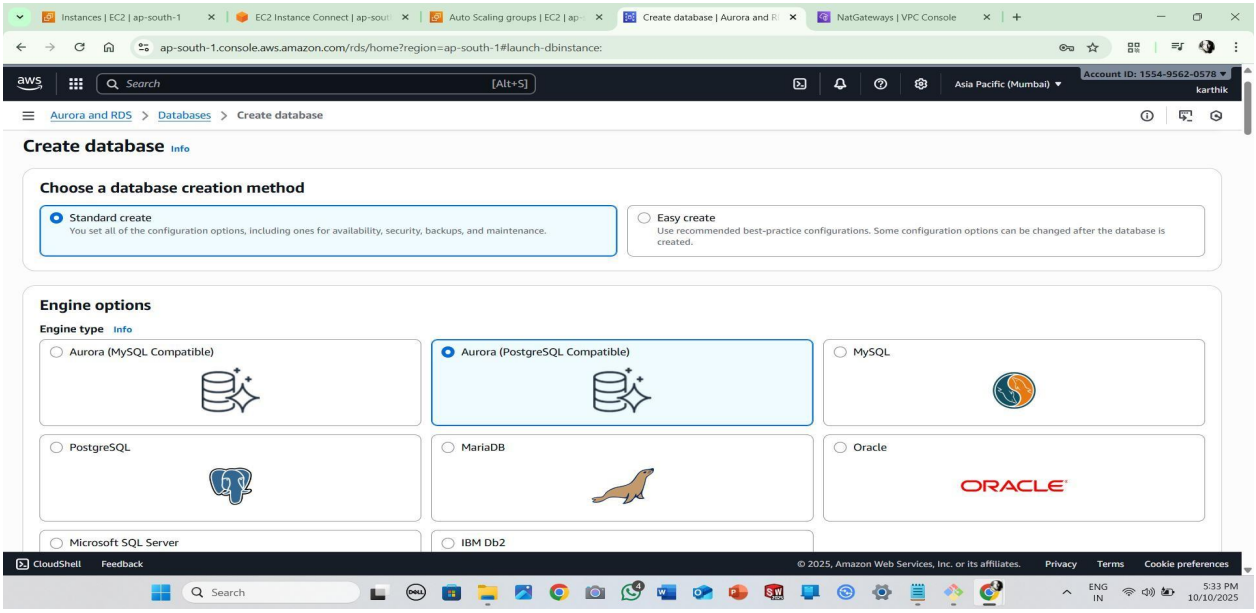


Fig-13

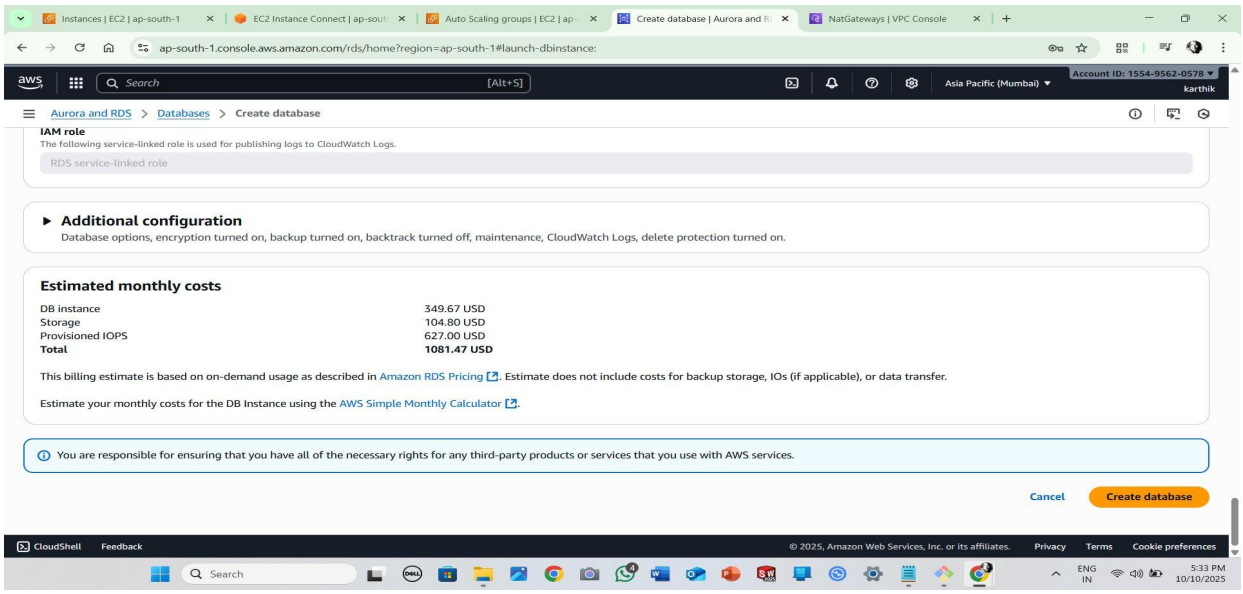


Fig-14

g) Add Read Replica to Database.

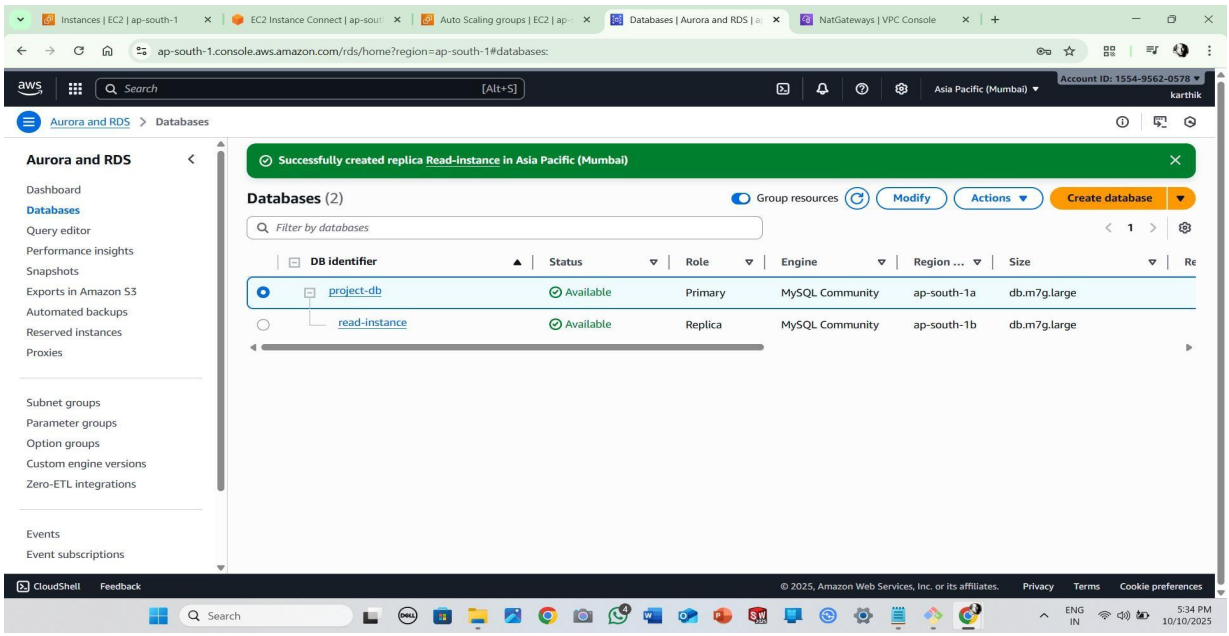


Fig-15

Step-11: Create subnet in RDS:

a) Open rds and Create a Subnets Group.

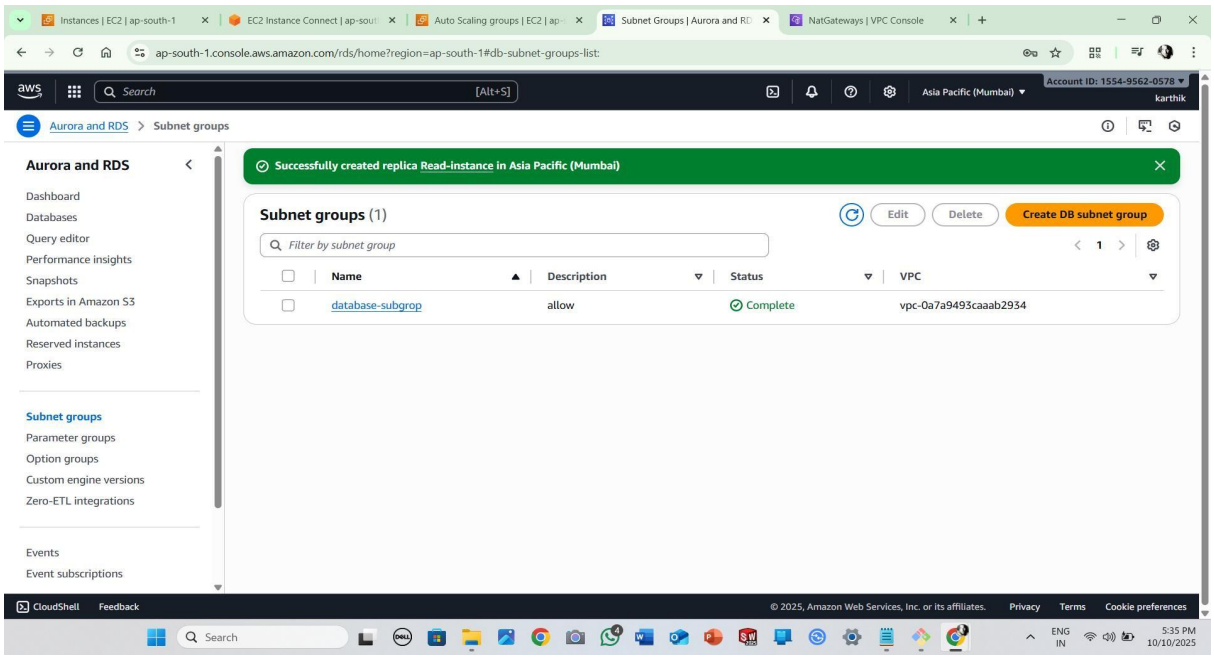


Fig-16

Step-12: Create MySQL:

a) Open MySQL Workbench

b) In MySQL open Database.

c) Give host name and Password to it.

d) TO Create Table and Insert Data.

BY Using Commands, we have to insert Data.

a) show databases;

b) create database Devops;

c) 3.CREATE TABLE details (

d) ID Int NOT NULL;

e) Firstname varchar (255);

f) Lastname varchar(255);

g) age int);

h) show tables;

i) INSERT INTO details (ID, Firstname, Lastname, age)

j) select * from details;

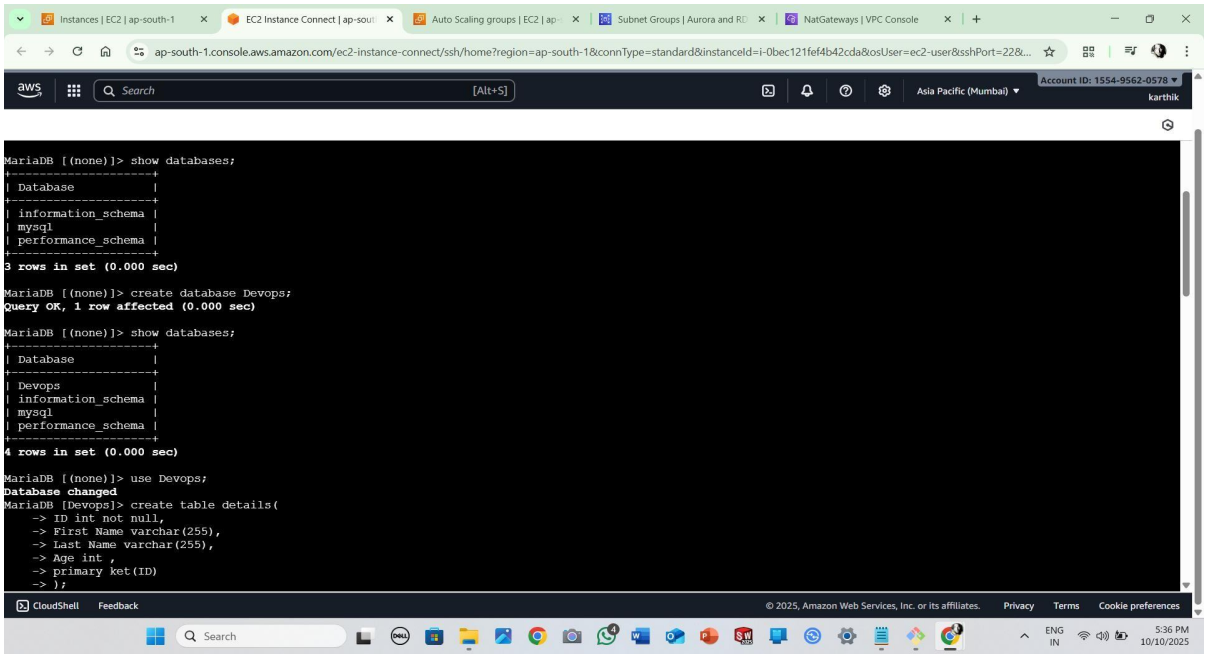


Fig-17

```
MariaDB [Devops]> CREATE TABLE details (
    -> ID INT NOT NULL,
    -> Firstname VARCHAR(255),
    -> Lastname VARCHAR(255),
    -> Age INT
    -> );
Query OK, 0 rows affected (0.008 sec)

MariaDB [Devops]> show tables;
+------------------+
| Tables_in_Devops |
+------------------+
| details          |
+------------------+
1 row in set (0.000 sec)

MariaDB [Devops]> show * from details;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near '* from det
ails' at line 1
MariaDB [Devops]> describe details;
+-----------+--------------+------+-----+---------+-------+
| Field     | Type         | Null | Key | Default | Extra |
+-----------+--------------+------+-----+---------+-------+
| ID        | int(11)      | NO   |     | NULL    |       |
| Firstname | varchar(255) | YES  |     | NULL    |       |
| Lastname  | varchar(255) | YES  |     | NULL    |       |
| Age       | int(11)      | YES  |     | NULL    |       |
+-----------+--------------+------+-----+---------+-------+
4 rows in set (0.001 sec)

MariaDB [Devops]> INSERT INTO details (ID, Firstname, Lastname, Age)
    -> VALUES
    -> (1, 'Karthik', 'Reddy', 22),
    -> (2, 'Saketh', 'Kumar', 22);
```

Fig- 18

```
+------------------+
1 row in set (0.000 sec)

MariaDB [Devops]> show * from details;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near '* from det
ails' at line 1
MariaDB [Devops]> describe details;
+-----------+--------------+------+-----+---------+-------+
| Field     | Type         | Null | Key | Default | Extra |
+-----------+--------------+------+-----+---------+-------+
| ID        | int(11)      | NO   |     | NULL    |       |
| Firstname | varchar(255) | YES  |     | NULL    |       |
| Lastname  | varchar(255) | YES  |     | NULL    |       |
| Age       | int(11)      | YES  |     | NULL    |       |
+-----------+--------------+------+-----+---------+-------+
4 rows in set (0.001 sec)

MariaDB [Devops]> INSERT INTO details (ID, Firstname, Lastname, Age)
    -> VALUES
    -> (1, 'Karthik', 'Reddy', 22),
    -> (2, 'Saketh', 'Kumar', 22);
Query OK, 2 rows affected (0.002 sec)
Records: 2  Duplicates: 0  Warnings: 0

MariaDB [Devops]> select * from details;
+----+-----------+----------+------+
| ID | Firstname | Lastname | Age  |
+----+-----------+----------+------+
|  1 | Karthik   | Reddy    |   22 |
|  2 | Saketh    | Kumar    |   22 |
+----+-----------+----------+------+
2 rows in set (0.000 sec)

MariaDB [Devops]>
```

Fig-19

CONCLUSION

In this project, we successfully designed and deployed a 3-tier architecture on Amazon Web Services (AWS), comprising the Presentation Layer (Web Tier), Application Layer (App Tier), and Database Layer (DB Tier). This architecture enhances security, scalability, and manageability of applications.

a) Implemented VPC, public and private subnets, ensuring network isolation.

b) Configured EC2 instances for the web and application tiers.

c) Deployed a MySQL RDS instance in the private subnet for secure data storage.

d) Utilized Security Groups to manage fine-grained access control between layers.

e) Ensured high availability and scalability using Auto Scaling Groups and Elastic Load Balancer (ELB).

This architecture follows industry best practices for building cloud-native applications and can be scaled further to meet growing business needs. It demonstrates how AWS services can be combined effectively to create a secure, modular, and reliable infrastructure.